Day 88 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 5 reference

Watch: FastText: Solving the Out-of-Vocabulary Problem with Subword Embedding (Dr. Partha Majumdar) | FastText Explained: Subword Embeddings in NLP (Tech Pulse Labs)

`day88_subword_embeddings.py`, segment by segment, in the order `main()` uses things. Day 86's corpus, pair and scoring code is imported and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: G(w) = the rows of word w (its own row, then one per n-gram) · v_w = its centre vector · B = 200,000 n-gram rows

Segment 0 — Setup

Settings, and importing Day 86

CODE

```
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day86_word2vec_skipgram as d86
np.random.seed(42)
NMIN, NMAX = 3, 6
BUCKETS = 200_000
LR_FT = 0.05
HELD = {g: ws[::2] for g, ws in d86.CATEGORIES.items()
        if g in ("number", "month", "colour", "element", "instrument", "science")}
HELD_WORDS = {w for ws in HELD.values() for w in ws}
SUFFIXES = ("s", "es", "ed", "ing", "ly", "er", "ers", "ism", "ist", "ists", "ian", "ians", "al",
    "ic")
```

- `sys.path.insert(0, HERE)` — lets Python find Day 86's script in the same folder.
- `HELD = {g: ws[::2] for g, ws in d86.CATEGORIES.items() if g in (...)}` — a dict comprehension with a filter: every second word ([::2]) of six groups.
- `{w for ws in HELD.values() for w in ws}` — a set comprehension with two loops: all held-out words in one set.

Segment 1 — N-grams and hashing

char_ngrams, fnv1a, ngram_rows

MATH

$$h \leftarrow 2166136261; \quad \text{for each byte } b: h \leftarrow (h \text{ xor } b) \cdot 16777619 \bmod 2^{32}; \quad row = V + h \bmod B$$

CODE

```
def char_ngrams(word, nmin=NMIN, nmax=NMAX):
    w = "<" + word + ">"
    return [w[i:i + n] for n in range(nmin, nmax + 1) for i in range(len(w) - n + 1)]
def fnv1a(s):
    h = 2166136261
    for b in s.encode("utf-8"):
        h = ((h ^ b) * 16777619) & 0xFFFFFFFF
    return h
def ngram_rows(word, V):
    return [V + fnv1a(g) % BUCKETS for g in char_ngrams(word)]
```

- `[w[i:i + n] for n in range(nmin, nmax + 1) for i in range(len(w) - n + 1)]` — two loops in one list: every length, then every start position.
- `s.encode("utf-8")` — the string as bytes; iterating over bytes gives integers 0-255.
- `((h ^ b) * 16777619) & 0xFFFFFFFF` — ^ is bitwise xor; & 0xFFFFFFFF keeps the lowest 32 bits, as a 32-bit integer would.
- `V + fnv1a(g) % BUCKETS` — n-gram rows come after the V word rows in the same table.

Segment 2 — The model

FastText: a flat list of rows per word

MATH

$$v_w = (1 / |G(w)|) \Sigma_{g \text{ in } G(w)} W_{in}[g]$$

CODE

```
class FastText:
    def __init__(self, itos, d=d86.D, seed=0):
        V = len(itos)
        rng = np.random.RandomState(seed)
        self.Win = (rng.rand(V + BUCKETS, d) - 0.5) / d
        self.Wout = np.zeros((V, d))
        rows = [[i] + ngram_rows(w, V) for i, w in enumerate(itos)]
        self.cnt = np.array([len(r) for r in rows])
        self.start = np.cumsum(self.cnt) - self.cnt
        self.flat = np.array([x for r in rows for x in r])
    def gather(self, c):
        n = self.cnt[c]
        seg = np.cumsum(n) - n
        pos = np.arange(n.sum()) - np.repeat(seg, n) + np.repeat(self.start[c], n)
        return self.flat[pos], np.repeat(1.0 / n, n), seg, n
    def centre(self, c):
        rows, w, seg, _ = self.gather(c)
        return np.add.reduceat(self.Win[rows] * w[:, None], seg, axis=0)
```

- `self.flat = np.array([x for r in rows for x in r])` — every word's row list, one after another; start and cnt say where each word's part is.
- `np.repeat(seg, n)` — each segment's start, repeated once per row in it.

- `np.add.reduceat(A, seg, axis=0)` — sums A's rows segment by segment: one sum per word, in one call.

FastText.step and oov

MATH

$$dW_{in}[g] = dv_{w} / |G(w)| \text{ for every } g \text{ in } G(w)$$

CODE

```
def step(self, c, tgt, lr):
    rows, w, seg, n = self.gather(c)
    v = np.add.reduceat(self.Win[rows] * w[:, None], seg, axis=0)
    U = self.Wout[tgt]
    z = np.einsum("nd,nkd->nk", v, U)
    loss = np.logaddexp(0, -z[:, 0]).sum() + np.logaddexp(0, z[:, 1:]).sum()
    gz = d86.sigmoid(z); gz[:, 0] -= 1
    dv = np.einsum("nk,nkd->nd", gz, U)
    np.add.at(self.Win, rows, -lr * w[:, None] * np.repeat(dv, n, axis=0))
    np.add.at(self.Wout, tgt, -lr * gz[:, :, None] * v[:, None, :])
    return loss
def vectors(self):
    return np.concatenate([self.centre(np.arange(s, min(s + 4096, len(self.cnt)))) for s in
        range(0, len(self.cnt), 4096)])
def oov(self, word, V):
    return self.Win[ngram_rows(word, V)].mean(axis=0)
```

- `np.repeat(dv, n, axis=0)` — each word's gradient copied once per row of that word.
- `np.add.at(self.Win, rows, -lr * w[:, None] * ...)` — every row gets its share 1/n; a row shared by many words in the batch collects them all.
- `self.Win[ngram_rows(word, V)].mean(axis=0)` — an unseen word: only its n-gram rows exist.

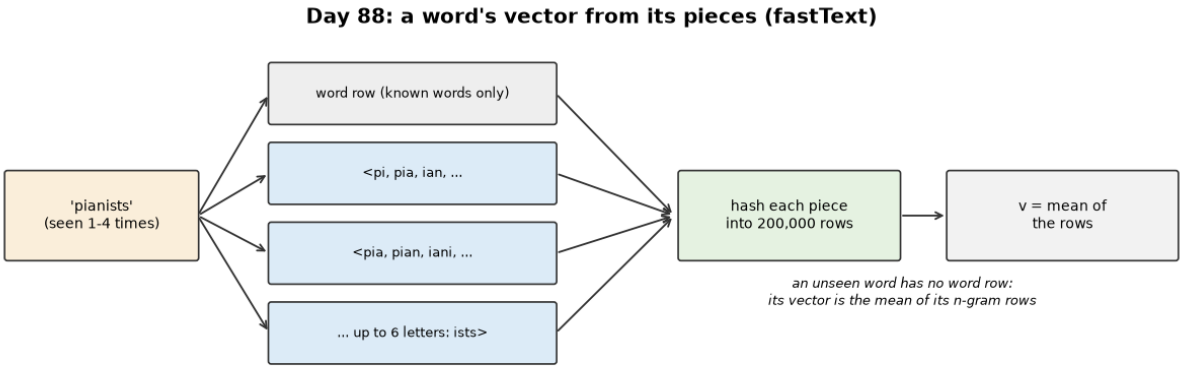
train: Day 86's loop, with the model passed in

CODE

```
def train(model, ids, freq, lr0, seed, scorer):
    rng = np.random.RandomState(seed + 100)
    cdf = d86.noise_cdf(freq)
    total, done, hist = d86.EPOCHS * len(ids), 0, []
    for ep in range(d86.EPOCHS):
        t0 = time.perf_counter()
        kept = d86.subsample(ids, freq, d86.SUBSAMPLE, rng)
        C, O = d86.skipgram_pairs(kept, d86.WINDOW, rng)
        perm = rng.permutation(len(C)); C, O = C[perm], O[perm]
        for s in range(0, len(C), d86.BATCH):
            c, o = C[s:s + d86.BATCH], O[s:s + d86.BATCH]
            lr = lr0 * max(1e-4, 1 - (done + s / len(C) * len(ids)) / total)
            tgt = np.concatenate([o[:, None], np.searchsorted(cdf, rng.rand(len(c), d86.NEG))],
                axis=1)
            model.step(c, tgt, lr)
        done += len(ids)
        hist.append((time.perf_counter() - t0,) + scorer(model))
    return np.array(hist)
```

- `scorer(model)` — a function given as an argument: each model is scored its own way (Win or vectors()).
- `d86.subsample(...), d86.skipgram_pairs(...)` — Day 86's own functions, called through the imported module.

SYSTEM DIAGRAM



A word's vector from its pieces.

Segment 3 — Measures

held_check and jaccard

MATH

$$Jaccard(a, b) = |G(a) \cap G(b)| \,/\, |G(a) \cup G(b)|$$

CODE

```
def held_check(sim_of, cand, cand_groups):
    ok = []
    for g, ws in HELD.items():
        for w in ws:
            ok.append(cand_groups[int(np.argmax(sim_of(w)))] == g)
    return np.mean(ok)
def jaccard(a, b):
    A, B = set(char_ngrams(a)), set(char_ngrams(b))
    return len(A & B) / len(A | B)
```

- `cand_groups[int(np.argmax(sim_of(w)))]` — the group of the most similar candidate.
- `len(A & B) / len(A | B)` — set intersection over union: the spelling-only similarity.

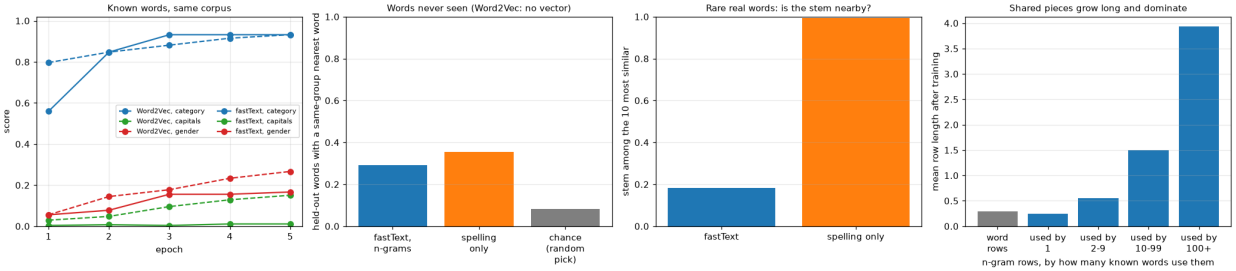
demo_why: row lengths by sharing

CODE

```
use = Counter(r for i in range(V) for r in set(ft.flat[ft.start[i] + 1:ft.start[i] +
    ft.cnt[i]].tolist()))
rows = np.array(list(use)); cnt = np.array([use[r] for r in rows])
norms = np.linalg.norm(ft.Win[rows], axis=1)
```

- `set(ft.flat[ft.start[i] + 1:ft.start[i] + ft.cnt[i]].tolist())` — word i's n-gram rows (its own row skipped), each counted once.
- `Counter(r for i in ... for r in ...)` — a generator with two loops, counted: how many words use each row.

REAL GRAPH



Real graph from the script: known words, words never seen, rare words, and row lengths.

Quick Syntax Glossary — new or reinforced constructs

- `a ^ b, a & mask` — bitwise xor and bitwise and.
- `s.encode("utf-8")` — text to bytes.
- `np.add.reduceat(A, starts, axis=0)` — sum rows segment by segment.
- `np.repeat(a, counts)` — each entry repeated its own number of times.
- `{g: ws[::2] for g, ws in d.items() if ...}` — a filtered dict comprehension.
- `global NAME` — assign to a module-level name inside a function (used to shrink the table for the check).

End of Day 88 syntax reference. For the experiments and what they showed, see the Day 88 complete guide.